OpenRCA 2.0: From Outcome Labels to Causal Process Supervision

Aoyang Fang **Yifan Yang** **Jin'ao Shang*** **Qisheng Lu**
Rui Wang Junjielung Xu Haoran Tang† Songhan Zhang
Pinjia He‡*
The Chinese University of Hong Kong, Shenzhen
{aoyangfang, yifanyang6, qishenglu, 224040299, junjielongxu, 222010549}@link.cuhk.edu.cn
*jinao_s@stu.xjtu.edu.cn †tanghaoran66666@gmail.com hepinjia@cuhk.edu.cn

Abstract

Root cause analysis poses a holistic test of LLM software capabilities, such as long-context understanding, multi-step reasoning, and tool use. However, existing datasets suffer from a fundamental gap: they lack evidence of fault propagation, which largely simplifies the task to naive pattern matching. To support rigorous evaluation, we introduce FORGE, a step-wise labeling system that leverages known interventions from fault injection to reconstruct causal propagation paths via forward verification (reasoning from cause to effect rather than inferring backward from symptoms) and provides step-wise supervision for each failure instance. With FORGE, we construct OpenRCA 2.0, the first RCA benchmark that provides step-wise process supervision for LLM agents. Evaluation on 7 LLMs reveals that even the most advanced model, while achieving 0.76 Pass@1 accuracy, produces a valid causal path from root cause to symptom in only 0.63 of cases; averaged across all models, this drops to 0.43. These findings demonstrate that outcome-based evaluation alone masks potential reasoning deficiencies, and that process-level evaluation is essential for trustworthy diagnostic LLM agents.

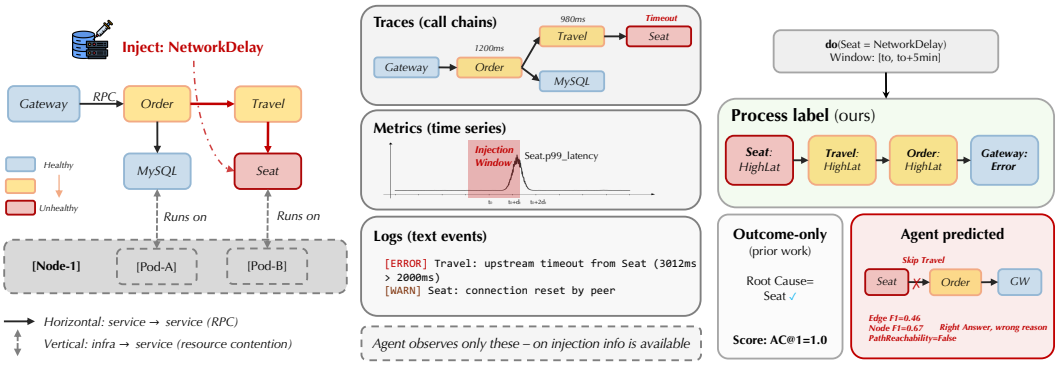
1 Introduction

Root cause analysis (RCA) in complex software systems serves as a holistic testbed for AI systems, requiring integrated capabilities in comprehension, multi-step reasoning, long-horizon planning, and tool use Xu et al. [2025], Chen et al. [2025b,a], Zhang et al. [2025a]. In this setting, there are numerous loosely coupled services communicating via network calls Zhou et al. [2018]. Once a fault occurs in a certain service, it may cascade along the call dependencies and trigger faults in distant components (Figure 1a), making it difficult to diagnose the root cause. The agent is expected to diagnose the root cause across a large volume of system telemetry, including distributed traces, time-series metrics, and logs (Figure 1b). This requires retrieving relevant signals, forming and revising hypotheses, and reasoning over noisy propagation patterns to identify the small set of faults that triggered cascading failures across services.

However, existing RCA benchmarks evaluate only the *outcome* ("Did the agent name the right root cause?") while ignoring the *process* ("Did it correctly trace how the fault propagated?"), as illustrated in Figure 1c. Specifically, they usually collect the data via *fault injection* Pham et al. [2025], Fang et al. [2025], Chen et al. [2025b], treating the injected component as the final ground truth. However, when an agent outputs the correct answer, it does not always reflect sound reasoning, as a flawed derivation process may also happen to reach the ground truth via random guessing Xu et al. [2025]. Furthermore,

*‡Corresponding author.

Preprint.



(a) Microservice System & Fault Propagation (b) Observable Telemetry (c) Outcome vs. Process Evaluation

Figure 1: Overview. **(a)** Microservice systems form a two-layer dependency graph: services communicate via RPC (horizontal edges, e.g., `Gateway Service` call `Order Service`), while infrastructure resources are shared across co-located services (vertical edges, e.g., `Seat Service` is deployed on `Pod-B`). Injecting a NetworkDelay fault into `Seat` triggers a cascade along the RPC chain. **(b)** An RCA agent observes three telemetry modalities (traces, metrics, and logs), but has no access to the injection information. **(c)** Prior benchmarks provide only an outcome label (*Root Cause = Seat*). OpenRCA 2.0 provides the full causal propagation chain with state-level annotations (process-level), reconstructed by FORGE, revealing that agents often produce the correct root cause through spurious causal reasoning.

a recent study shows that such outcome-oriented methods may introduce "silent" faults (invisible to users) or "singleton" faults (faults that exist only in the injected service), and the detection method is naive Fang et al. [2025]. Without process-level ground truth, there is no principled way to assess whether an agent's diagnosis reflects genuine causal reasoning or a lucky guess that happens to match the injected component Xu et al. [2025]. *In practice, a correct root cause accompanied by flawed reasoning can mislead engineers into investigating irrelevant components, increasing resolution time rather than reducing it.*

To bridge this gap, we propose *FORGE*, a framework that automatically reconstructs verified causal chains from fault injection experiments. Our key insight exploits an information asymmetry: RCA is inherently a *backward* problem, reasoning from symptoms to an unknown cause, whereas our annotation proceeds via *forward verification*, reasoning from cause to effect, since the injected fault, its location, and timing are known a priori Pearl [2009]. This asymmetry converts an ill-posed inverse problem into a well-posed verification task that is strictly easier than the diagnostic problem it evaluates, enabling automated recovery of step-wise causal propagation paths without human annotation.

With FORGE, we construct OpenRCA 2.0 by recovering verified causal paths from 8137 fault injections Fang et al. [2025], adding process-level ground truth to previously outcome-only instances. The results reveal a pervasive gap between diagnostic accuracy and causal reasoning quality: the best agent (Claude-4.5-Sonnet) achieves 0.76 Pass@1 accuracy for root cause identification, yet only 0.63 Path Reachability. Roughly one in six correct diagnoses lack a valid causal path to any observed symptom. Averaged across all 7 models, the gap widens: Pass@1 drops to 0.52 with only 0.43 Path Reachability. Agents further hallucinate an average of 2.1 non-existent causal edges per diagnosis.

Our contributions are:

- **A process-level oracle machine via forward verification.** We propose FORGE, which exploits the interventional nature of fault injection to automatically reconstruct verified causal propagation paths through a coarse-to-fine verification pipeline.
- **An RCA benchmark with step-wise supervision.** We introduce OpenRCA 2.0, comprising 500 instances with verified causal annotations for reproducible evaluation. We further introduce metrics that evaluate not just *what* root cause an agent identifies, but *how* it arrives at the diagnosis.

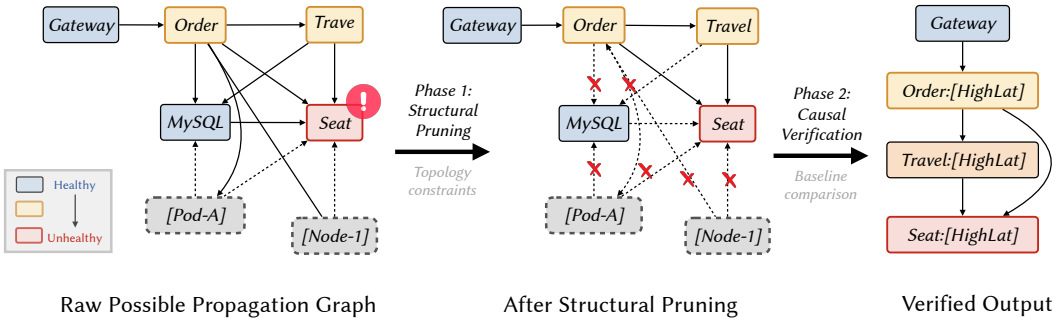


Figure 2: Coarse-to-Fine Verification pipeline. FORGE transforms raw fault injection records into verified causal propagation paths through two phases. *Phase 1 (Structural Pruning)* projects telemetry onto discrete meta-type-aware states, applies topological constraints via a propagation rule set, and generates candidate paths through time-expanded search on a heterogeneous graph. *Phase 2 (Causal Verification)* profiles a pre-injection baseline and prunes coincidental anomalies via baseline-relative deviation scoring; injections with no surviving path are discarded as silent. The output is a set of verified, difficulty-stratified causal paths serving as process-level ground truth.

- **Empirical evidence of reasoning deficiencies.** Outcome-based metrics alone mask potential reasoning deficiencies, with pervasive hallucination of non-existent propagation paths across all tested models.

2 FORGE: Constructing Process-Level Ground Truth

We now describe the FORGE pipeline (Figure 2). Given fault injection records and the system dependency graph, it reconstructs verified causal propagation paths through two phases: *Structural Pruning* (Section 2.3) eliminates topologically impossible propagation sequences to produce a candidate path set, and *Causal Verification* (Section 2.4) compares injection-period telemetry against a pre-injection baseline to retain only statistically significant fault-induced chains. We first introduce the domain concepts and formalism that underpin the pipeline.

2.1 Background

We model a microservice system as a directed dependency graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where nodes $\mathcal{V}$ are either *logical* (services such as `Order`, `Travel`) or *infrastructural* (Pods and Nodes, where each Pod is a runtime unit scheduled onto a physical or virtual Node by Kubernetes), and edges $\mathcal{E}$ encode invocation or resource dependencies (Figure 1a). This heterogeneous node typing gives rise to two distinct propagation channels that the pipeline must distinguish. *Horizontal propagation* follows RPC call chains between services: a slow `Order` service causes its upstream callers to time out, producing cascading latency spikes along the call graph. *Vertical propagation* crosses abstraction layers: CPU throttling on a Kubernetes node degrades all co-located services simultaneously, even those with no mutual RPC dependency. A single injection can trigger both channels, creating multi-hop paths that mix horizontal and vertical edges.

These channels leave distinct signatures in the three telemetry modalities, which the pipeline exploits for verification. *Traces* record per-request call trees with span-level latencies and status codes; horizontal propagation manifests as anomalous parent→child span timing that reveals the cascade direction. *Metrics* are per-service and per-host time series (CPU usage, p99 latency, error rate); vertical propagation appears as correlated anomalies among co-located services sharing a throttled resource. *Logs* are semi-structured text events (e.g., `[ERROR] upstream timeout`) that supply causal context absent from numerical signals alone. Section 2.4 describes how these modality-specific signatures are used to verify candidate propagation edges.

Fault injection provides the interventional anchor for this verification. In causal inference terms, the experimenter performs $do(v_{root}$=fault_type) during a controlled window $[t_0, t_0 + \Delta t]$ Pearl [2009], where the target, fault type, and timing are all known a priori. This known intervention converts the

annotation task from backward causal inference into forward verification, an asymmetry we formalize next.

2.2 Formal Problem Setup

We formalize the annotation task using the dependency graph and intervention semantics introduced above. We model fault propagation as a structural causal model (SCM) Pearl [2009] over discrete time steps, which specifies, for every component, how its health state is causally determined by the states of the components it depends on. Concretely, each node $v_i \in \mathcal{V}$ evolves according to:

$$S_i^{(t)} = f_i\big(\mathbf{S}_{Pa(v_i)}^{(<t)},\ \mathbf{H}_i^{(t)},\ U_i^{(t)}\big) \tag{1}$$

where f_i is the (unknown) causal mechanism governing component v_i, and $\mathbf{S}_{Pa(v_i)}^{(<t)}$ denotes the historical states of v_i's *causal parents* in $\mathcal{G}$ (i.e., the services or resources that directly influence v_i), reflecting that fault effects propagate with finite delay along service dependencies. $\mathbf{H}_i$ captures latent confounders (e.g., shared infrastructure load that simultaneously affects multiple co-located services), and U_i represents exogenous (externally driven) noise. Intuitively, Equation (1) says that a component becomes unhealthy only because something it depends on was already unhealthy, or because of an external shock; faults do not arise spontaneously. An RCA agent observes only telemetry $\mathbf{O}_{obs}$ and must jointly infer the dependency structure, root cause, and propagation path. In practice, even this observation is incomplete: instrumentation gaps, sampling policies, and misconfigured exporters mean that $\mathbf{O}_{obs} \subset \mathbf{O}_{true}$, leaving blind spots that further compound the difficulty. This is an inherently *backward* (abductive) problem: the agent must reason from observed symptoms back to the unobserved cause, complicated by the latent confounders $\mathbf{H}_i$, exogenous noise U_i, and partial observability. FORGE, by contrast, has access to the intervention $do(v_{root})$, giving it a strictly richer information set:

$$\underbrace{\{\mathbf{O}_{obs}\}}_{\text{Agent}} \subset \underbrace{\{\mathbf{O}_{obs},\ do(v_{root})\}}_{\text{FORGE}} \tag{2}$$

Here, $do(v_{root})$ denotes the complete intervention specification: the target component v_{root}, fault type (e.g., CPU stress, network partition), injection time window $[t_0, t_0{+}\Delta t]$, and fault parameters (e.g., CPU throttle percentage, delay magnitude). The agent, by contrast, receives only the telemetry $\mathbf{O}_{obs}$ partitioned into normal and abnormal periods, without knowing which component was injected, when the fault began, or what type of fault occurred. Because each fault type produces predictable telemetry signatures (e.g., CPU stress manifests as elevated CPU utilization; network delay as increased request latency), knowing the intervention enables pattern-directed verification rather than unconstrained search. This information asymmetry converts the ill-posed inverse problem (inferring the unknown root cause from symptoms) into a well-posed *forward* verification task (confirming that observed anomalies are consistent with the known intervention propagating through $\mathcal{G}$). The recovered graph $\mathcal{G}^*$ serves as the process-level ground truth against which agent reasoning is evaluated.

2.3 Phase 1: Structural Pruning

This phase eliminates structurally impossible propagation paths by enforcing physical and logical constraints from the system topology. It is motivated by the observation that LLMs often propose plausible-sounding but structurally impossible propagation paths Chen et al. [2025b]; the resulting ground truth therefore tests whether agents can adhere to topological constraints. The construction process involves three key steps:

Step 1: Meta-Type Aware Abstraction. We first project the high-dimensional raw telemetry $\mathbf{O}^{(t)} \in \mathbb{R}^d$ onto a semantically clear discrete state space Σ. We define a mapping function $\Psi(O_i, \tau(v_i)) \to s_i \in \Sigma$ that extracts significant abnormal states based on node type τ. For *Logical Nodes* (e.g., Services), states capture performance violations (e.g., `HighLatency`, `HighErrorRate`); for *Infrastructure Nodes* (e.g., Pods), states reflect resource exhaustion (e.g., `CPU_Throttled`, `MemoryPressure`). This abstraction filters out high-frequency noise (U_i), retaining only semantically meaningful events.

Step 2: Propagation Rule Filtering. We define a compact rule set $\mathcal{R}$ that enumerates the physically valid state transitions between adjacent nodes. Crucially, $\mathcal{R}$ is *fault-type-specific* rather than *system-specific*: each rule is derived from well-established failure propagation mechanisms in microservice

and container-orchestrated architectures (e.g., resource contention, RPC timeout cascading, shared-storage degradation), and applies to any system that exhibits the same fault type regardless of its particular service topology. Each rule takes the form $(\tau_i\colon s_i) \xrightarrow{\text{channel}} (\tau_j\colon s_j)$: for example, `Infra:CPU_Throttled` $\xrightarrow{\text{vertical}}$ `Logic:HighLatency` (resource contention degrades co-located services), or `Logic:HighLatency` $\xrightarrow{\text{horizontal}}$ `Logic:HighLatency` (upstream slowdown cascades to callers). *The rule set is intentionally conservative: it admits all mechanistically plausible transitions, so that false positives are deferred to Phase 2's statistical verification rather than silently dropped here.* Transitions not covered by $\mathcal{R}$ lack any known causal mechanism and are pruned, eliminating paths such as a service directly causing a node failure without an intermediate resource link. The full rule table and its derivation are provided in Appendix E.3.

Step 3: Candidate Path Generation. We construct a Time-Expanded Heterogeneous Graph where nodes are tuples (v, s, t). We employ a constrained Depth-First Search (DFS) to identify a set of candidate paths Π_{cand}. This search unrolls circular dependencies (common in microservices) into acyclic chains in time, ensuring that the generated candidates are topologically valid sequences of state transitions.

2.4 Phase 2: Causal Verification

This phase separates genuine fault propagation from coincidental background disturbances (e.g., periodic cron jobs) that co-occur with the injection window. The resulting annotations capture only causally verified paths, providing ground truth that tests whether agents can distinguish true causal mechanisms from spurious correlations.

Step 1: Baseline Profiling. For each fault injection experiment, we deploy a fresh microservice instance, apply a warm-up period to reach steady state, then collect normal-period telemetry to construct a *Reference Baseline* ($\mathcal{D}_{base}$) immediately before injection. This controlled experimental design ensures that $\mathcal{D}_{base}$ approximates the counterfactual distribution $P(\cdot \mid do(v_{root} = \emptyset))$ without confounding from deployment changes, traffic shifts, or prior incidents (detailed protocol in Appendix E.6).

Step 2: Baseline-Relative Anomaly Screening. We compare each node's behavior during the injection window $\mathcal{D}_{int}$ against the baseline $\mathcal{D}_{base}$ to separate fault-induced anomalies from background noise (U_i) and latent confounders ($\mathbf{H}_i$, e.g., shared infrastructure load, periodic maintenance tasks). For each node v_i along a candidate path $\pi \in \Pi_{cand}$, we apply distributional tests tailored to each metric type (e.g., Z-score for Gaussian metrics, percentile-based tests for heavy-tailed metrics; details in Appendix E.7). Importantly, the per-node anomaly test is designed as a *permissive screen* rather than a strict statistical test: individual thresholds are calibrated to minimize false negatives, because a missed anomaly permanently removes genuine propagation evidence from the candidate path. False positives, by contrast, are recoverable: the true filtering power comes from the *conjunction* of three independent conditions. A candidate path π is *verified* only if *all* of the following hold simultaneously: (i) every node along π exhibits a deviation beyond its baseline screen, (ii) every consecutive edge conforms to the propagation rule set $\mathcal{R}$ from Phase 1, and (iii) each downstream anomaly onsets no earlier than its upstream cause (temporal causality). Any single condition may admit false positives in isolation; their conjunction ensures that only paths with consistent statistical, structural, and temporal evidence survive.

Injections for which no path survives verification are classified as *silent*: the fault was absorbed by system resilience (e.g., auto-scaling, retries) and produced no observable failure. These samples are discarded to prevent label noise. The resulting dataset contains only verified, non-trivial failure propagation paths with step-wise annotations, enabling rigorous *process-level evaluation* of whether RCA agents perform genuine causal reasoning rather than exploiting surface-level correlations.

3 Experiments

We first describe the experimental setup (Section 3.1), then evaluate how process-level ground truth reveals reasoning gaps in LLM-based RCA agents (Section 3.2).

Table 1: Benchmark summary. We apply FORGE to fault injection instances from Fang et al. Fang et al. [2025], recovering verified causal paths and forming OpenRCA 2.0 (500 instances) for reproducible evaluation.

Property	**Value**
Target system	TrainTicket (44 services)
Cluster nodes	7
Fault types	25 (application + infrastructure)
Telemetry modalities	Traces, Metrics, Logs
Total injections	8137
Silent injections filtered	39.9%
Evaluation instances (with causal paths)	500
Propagation depth (SPL)	1–5+ (median 3)
Avg. causal edges per instance	7.0

3.1 Experimental Setup

System and Faults. We use TrainTicket Zhou et al. [2018], a microservice benchmark comprising 44 services, and adopt the publicly released fault injection dataset from Fang et al. Fang et al. [2025]. The dataset comprises 8137 injections across 25 distinct fault types spanning application-level faults (HTTP delays, JVM exceptions, database latency) and infrastructure-level faults (CPU stress, network partition, pod kill); full taxonomy in Appendix B.2. Of these, 39.9% are classified as *silent*, absorbed by system resilience with no observable downstream effect Fang et al. [2025]. We apply the FORGE pipeline (Section 2) to the non-silent instances, recovering verified causal propagation paths and forming OpenRCA 2.0 (500 instances). All experimental results in this paper are reported on this set. Table 1 summarizes the resulting benchmark.

Ground Truth. Causal propagation paths are extracted automatically using the pipeline described in Section 2. Each extracted path is independently validated by an LLM-based verification agent. Unlike the RCA agents under evaluation, which must infer the root cause from symptoms alone, the verification agent receives the full intervention specification and operates in a forward verification setting. Given this privileged information, the agent uses tool-augmented reasoning to examine telemetry evidence for each edge along the path, providing an independent cross-check on the extraction pipeline (protocol details in Appendix E.4).

Agent Design. All evaluated agents follow a tool-augmented ReAct-style Yao et al. [2022] architecture, adopting the state-of-the-art Deep Research agent Lin [2025]. To isolate LLM reasoning capability from tooling effects, all models use the same single SQL tool with identical prompt templates; no explicit dependency graph is provided. Full architecture and prompt details are provided in Appendix C.

3.2 Benchmark Utility: Process-Level Evaluation of RCA Agents

Experimental Design. We evaluate the RCA agent architecture described above with seven state-of-the-art LLMs as backbones: Claude Sonnet 4.5, GPT-5.1, K2, GLM-4.7-Flash, Seed 1.6, Qwen3-Next-80B, and Qwen3-32B. Each model receives multi-modal telemetry (traces, metrics, logs) and must diagnose the root cause while explaining the failure propagation as a causal graph.

Evaluation Metrics. We evaluate agents along three axes (formal definitions in Appendix D). *Pass@1 Accuracy* measures whether the agent correctly identifies a true root cause on its first attempt. *Path Reachability* (PR) measures the fraction of all instances where the agent both identifies the correct root cause and constructs at least one valid directed path from it to a ground-truth symptom node, i.e., a service exhibiting suspicious SLO violations during the injection window (detailed criteria in Appendix D). PR is strictly upper-bounded by Pass@1; the gap between them directly quantifies how often correct diagnoses lack actionable propagation paths. Beyond diagnostic accuracy, we evaluate the quality of the agent's predicted causal graph $\hat{\mathcal{G}}$ against the ground-truth propagation path $\mathcal{G}^*$ using node and edge *precision*, *recall*, and *F1*. Node metrics treat each component in the graph as a prediction; edge metrics treat each directed causal link likewise. All graph-based metrics are computed on directed graphs by exact matching between the predicted graph $\hat{\mathcal{G}}$ and ground truth

Table 2: Process-level evaluation of the RCA agent with different LLM backbones on OpenRCA 2.0 (n=500). *Diagnostic Performance*: Pass@1 measures root cause identification accuracy; PR (Path Reachability) measures the fraction of instances where the agent both identifies the correct root cause and constructs a valid propagation path to a symptom (PR $\leq$ Pass@1). *Causal Graph Quality*: Edge and Node columns report F1 / Precision / Recall against ground truth; Halluc. counts fabricated causal edges per diagnosis. *Stratified Edge F1*: Edge F1 reported separately for correctly (+) and incorrectly ($-$) diagnosed cases.

	Diagnostic Performance		Causal Graph Quality			Stratified Edge F1	
Model	Pass@1↑	PR↑	Edge F1/P/R↑	Node F1/P/R↑	Halluc.↓	EF1 (+)↑	EF1 ($-$)
Claude Sonnet 4.5	0.76	0.63	0.53 / 0.75 / 0.46	0.69 / 0.83 / 0.63	1.3	0.59	0.35
GPT-5.1	0.63	0.53	0.46 / 0.54 / 0.49	0.71 / 0.72 / 0.78	3.8	0.51	0.39
K2	0.60	0.53	0.47 / 0.68 / 0.40	0.67 / 0.81 / 0.62	1.7	0.57	0.31
GLM-4.7-Flash	0.53	0.37	0.26 / 0.33 / 0.24	0.63 / 0.70 / 0.62	3.4	0.35	0.15
Seed 1.6	0.43	0.41	0.42 / 0.64 / 0.35	0.62 / 0.79 / 0.56	1.2	0.61	0.28
Qwen3-Next-80B	0.39	0.34	0.28 / 0.48 / 0.23	0.58 / 0.76 / 0.50	1.8	0.46	0.17
Qwen3-32B	0.32	0.18	0.18 / 0.33 / 0.14	0.52 / 0.79 / 0.41	1.4	0.30	0.13

$\mathcal{G}^*$; when multiple paths are predicted, we evaluate their union. Precision penalizes hallucinated components or spurious causal links, while recall penalizes missed parts of the true propagation path. We further stratify Edge F1 by diagnostic correctness, revealing whether correct diagnoses are supported by sound causal reasoning or merely coincidental. We use a deliberately sensitive symptom definition for SLO violations to avoid false negatives in propagation verification. A strict threshold would incorrectly classify many genuinely propagated faults as silent, making path verification impossible. Importantly, these thresholds are used only for ground-truth construction; agents do not observe the thresholds or detection labels, and must infer symptom services directly from raw telemetry.

Results. Table 2 presents the process-level evaluation. The two leftmost metric columns capture the core finding: Pass@1 measures whether the agent identifies the correct root cause, while PR additionally requires a valid causal path from that root cause to an observed symptom. The gap between them directly quantifies how often correct diagnoses rest on spurious causal reasoning.

(1) Correct diagnoses frequently lack valid causal reasoning. Claude Sonnet 4.5 achieves the highest Pass@1 (0.76) but its PR drops to 0.63, meaning that roughly one in six correct diagnoses cannot be traced through a valid propagation path. This gap is universal: across all seven models, PR is consistently and substantially lower than Pass@1. For an SRE, a correct root cause without a trustworthy and actionable propagation path offers limited value for remediation, as it provides no guidance on which intermediate services are affected.

(2) Agents infer root causes from partial evidence, not full causal reconstruction. The stratified Edge F1 columns reveal why the gap exists. Even in correctly diagnosed cases, the best model achieves only 0.59 Edge F1 against ground-truth causal edges (Edge F1 (+) for Claude). Agents appear to identify root causes through surface-level correlations or "shortcuts" rather than systematically tracing the propagation path. The Edge F1 ($-$) column confirms this interpretation: incorrectly diagnosed cases show markedly lower Edge F1 (e.g., 0.35 for Claude), indicating that reasoning quality and diagnostic accuracy are correlated but far from sufficient.

(3) Hallucination amplifies the reasoning gap. Fabricated causal edges inflate the predicted graph without improving reasoning validity. GPT-5.1 illustrates this clearly: despite the second-highest Pass@1 (0.63), it generates 3.8 hallucinated edges per diagnosis, nearly triple that of Claude (1.3). This drives its edge precision down to 0.54 (vs. Claude's 0.75) and widens the Pass@1–PR gap, as fabricated paths do not satisfy the reachability criterion.

(4) Process metrics differentiate models that outcome metrics cannot. Seed 1.6 and Qwen3-Next-80B achieve comparable Pass@1 (0.43 vs. 0.39), yet their reasoning quality diverges sharply: Seed attains Edge F1 of 0.42 with 1.2 hallucinated edges, while Qwen3-Next-80B reaches only 0.28 with 1.8. Consequently, Seed's PR (0.41) exceeds Qwen3-Next-80B's (0.34). Outcome-only evaluation would treat these models as near-equivalent; process-level metrics reveal that Seed produces substantially more faithful causal reasoning. A detailed case study of this phenomenon is presented in Section 3.3.

Table 3: Performance by propagation depth (SPL). All metrics are averaged across 7 models. Longer chains degrade both diagnostic accuracy and reasoning quality.

SPL	n	Pass@1	PR	Edge F1
2	152	65%	54%	41%
3	228	53%	43%	37%
4	94	38%	30%	34%
$\geq$5	21	25%	18%	29%

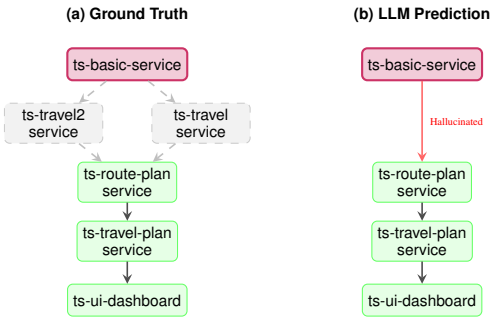


Figure 3: Spurious causal reasoning case. (a) Ground truth shows fault propagation through intermediate services (`travel`, `travel2`). (b) Claude correctly identifies root cause and symptoms but hallucinates a direct edge, missing the branching structure (Edge F1=40.0%).

(5) Performance degrades with propagation path length. Table 3 stratifies performance by the shortest propagation length (SPL), i.e., the minimum number of service hops from root cause to symptom. Pass@1 drops from 65% at SPL = 2 to 25% at SPL $\geq$ 5, and PR follows a parallel decline (from 54% to 18%), confirming that the reasoning gap widens as the ground-truth propagation chain grows longer, rather than being concentrated in short-chain cases.

3.3 Qualitative Analysis

Case Study: Spurious Causal Reasoning. Figure 3 presents a case study. The ground truth propagation path originates from a fault in ts-basic-service (the root cause). This fault propagates through two parallel intermediate services, ts-travel2-service and ts-travel-service, before impacting ts-route-plan-service, ts-travel-plan-service, and ultimately causing an SLO violation in the user-facing ts-ui-dashboard. The agent (Claude) correctly identifies ts-basic-service as the root cause (Root Cause F1 = 100%) and correctly pinpoints the downstream symptom nodes (route-plan, travel-plan, ui). However, the generated causal graph simplifies the propagation path: it hallucinates a direct link from ts-basic-service to ts-route-plan-service, completely skipping the crucial intermediate nodes ts-travel2-service and ts-travel-service (Node Recall = 57.1%). Consequently, while the diagnosis appears correct at the root cause level, the structural understanding is flawed, reflected in the low Edge Recall of 28.6% and Edge F1 of 40.0%. This case exemplifies why process-level ground truth is essential: despite a perfect root cause identification, the reasoning lacks critical dependencies, potentially hindering effective remediation strategies that rely on intermediate nodes.

4 Related Work

4.1 From Classification-Based Ranking to Generative Reasoning

The formulation of Root Cause Analysis (RCA) has undergone a fundamental shift. Traditionally, RCA was cast as a *multi-class classification problem*, where each candidate component is scored and ranked by its likelihood of being the root cause, using heuristic algorithms (e.g., Random Walk Yu et al. [2021]) or supervised graph neural networks Zhang et al. [2025b]. While effective for locating faults, these "black-box" approaches lack interpretability and fail to capture the sequential propagation mechanism. With the advent of Large Language Models (LLMs), recent works have pivoted towards

a *generative reasoning paradigm* Xu et al. [2025], Chen et al. [2025b,a], Zhang et al. [2025a], treating RCA as a causal narrative generation task. However, a critical misalignment persists: while the model architecture has evolved to handle complex reasoning, the underlying data supervision remains stagnant. Existing benchmarks Pham et al. [2025], Fang et al. [2025], Chen et al. [2025b] still rely on outcome-based labels that record only *which* component is the root cause, without annotating *how* the fault propagates, inherited from the classification era. This forces reasoning agents to optimize for the final answer directly, bypassing the logical derivation steps and encouraging the memorization of spurious correlations rather than learning robust causal dynamics.

4.2 Causal Structure Learning under Intervention

Our work intersects with causal discovery, particularly in dynamic systems. Classical observational methods (e.g., PC algorithm Spirtes et al. [2000]) struggle to recover Directed Acyclic Graphs (DAGs) in distributed systems due to unobserved confounders and high-dimensional noise Ikram et al. [2022], Han et al. [2025]. Pearl's hierarchy of causation posits that interventional data ($do(x)$) is essential for distinguishing true causality from mere correlation Pearl [2009]. While some RCA methods attempt to infer causal graphs relying solely on observational telemetry Li et al. [2022], Yu et al. [2023], they lack the *identifiability* guarantees that controlled interventions provide. FORGE distinguishes itself by adopting an *interventional* rather than purely observational perspective during annotation. While prior benchmarks use only the identity of the injected component as the ground-truth label, FORGE leverages the full intervention specification ($do(v_{root})$) to actively verify propagation paths. This effectively transforms the intractable structure learning problem into a tractable verification task, filtering out the "silent injections" and confounding noise that plague purely observational datasets.

4.3 Process Supervision and Verifiability

In the broader LLM landscape, focus is shifting from outcome-based supervision (Outcome Reward Models, ORMs) to *Process Supervision* (Process Reward Models, PRMs) Lightman et al. [2023], Uesato et al. [2022], Zeng et al.. Research in mathematical reasoning and code generation demonstrates that providing step-by-step verification signals significantly reduces hallucinations and improves generalization Yin et al. [2025], Yan et al. [2025]. However, applying this to system diagnostics is hindered by the *verification gap*: unlike math (where proofs are deterministic) or code (where unit tests exist), operational faults lack a natural "compiler" to verify reasoning steps. Current RCA benchmarks like AIOpsLab Chen et al. [2025b] or OpenRCA Xu et al. [2025] rely on human-curated scenarios or heuristically filtered logs, which are hard to scale and lack granular process labels. FORGE addresses this verification gap for system diagnostics by synthesizing verifiable failure propagation paths rooted in system topology and causal interventions, producing the first large-scale process-level ground truth for *evaluating* diagnostic reasoning in complex systems. The resulting step-wise annotations also open a path toward training PRMs in the reliability domain, though we leave this exploration to future work.

5 Discussion and Future Work

A controlled proxy for real-world diagnostics. Our experimental setting deliberately provides agents with complete, well-structured observability data: full distributed traces, time-aligned metrics, and structured logs. Notably, agents are *not* provided with the service dependency graph and must recover inter-service dependencies from trace data. The only information withheld is the injected fault itself. This represents an idealized upper bound on agent performance. In production environments, telemetry is frequently incomplete (services may lack instrumentation), noisy (metrics contain unrelated fluctuations), or heterogeneous (different teams adopt different logging formats and granularities). Source code, which often contains critical diagnostic signals such as error-handling logic and retry policies, is not yet included in our evaluation. The substantial reasoning gaps observed under these favorable conditions (Section 3.2) suggest that real-world performance would be considerably lower, making spurious causal reasoning even more prevalent. Crucially, full observability during dataset construction is a prerequisite for producing verified causal ground truth; once high-quality labels are established, we can systematically *subtract* information (e.g., dropping trace spans, masking metrics) to increase diagnostic difficulty while preserving label validity. We

plan to complement the automated verification with a human audit on a stratified random sample of verified paths to quantify residual annotation error.

Scope and generalization. The current benchmark is built on a single target system (TrainTicket, 44 microservices) and evaluates agents under a uniform ReAct-style architecture. While TrainTicket is a widely adopted benchmark in the AIOps community Zhou et al. [2018], its architecture (synchronous REST calls, relational databases) does not cover event-driven, service-mesh, or serverless patterns where failure propagation follows fundamentally different dynamics. Similarly, the ground-truth causal graphs capture propagation paths observable through telemetry signals; causal relationships mediated by shared resources (e.g., connection pool exhaustion, garbage collection pauses) may remain invisible at the current instrumentation granularity. Extending the benchmark to diverse system architectures and agent designs is an important direction for establishing broader generalizability. Notably, the propagation rule set $\mathcal{R}$ is bound to fault types and their underlying physical mechanisms (e.g., resource contention, RPC timeout cascading), not to any particular system topology, so it transfers directly to new microservice systems without re-engineering.

Ongoing extensions. We are actively expanding the benchmark along several axes to better approximate production complexity. First, we are introducing *concurrent and cascading fault* scenarios, where multiple faults interact and produce compounding propagation paths that are significantly harder to disentangle. Second, we are developing *code-level semantic faults* (e.g., off-by-one errors, incorrect business logic) that produce subtle behavioral deviations without clear telemetry anomalies, testing whether agents can reason beyond statistical signal detection. Third, we are incorporating additional microservice systems of varying scale and architectural style to assess cross-system generalization. Finally, we plan to introduce *degraded observability* settings where telemetry is partially missing or delayed, bridging the gap between our current idealized proxy and real-world operational conditions. Beyond evaluation, the step-wise causal annotations naturally lend themselves to process reward modeling (PRM) Lightman et al. [2023], Luo et al. [2024]: each verified propagation edge can serve as a supervision signal for training agents that produce faithful reasoning chains, not just correct final answers.

6 Conclusion

We presented FORGE, a framework that automatically reconstructs verified causal propagation paths from fault injection experiments. Applying it to 8137 injections from prior work Fang et al. [2025] recovers process-level ground truth for previously outcome-only instances, forming OpenRCA 2.0 (500 instances), the first RCA benchmark with step-wise causal annotations. Our approach exploits the information asymmetry between RCA agents, which must reason backward from symptoms, and annotation, which proceeds via forward verification from known interventions. Evaluating 7 LLMs reveals a consistent gap between diagnostic accuracy and causal reasoning quality: correct root cause identifications are frequently unsupported by valid causal paths, and agents hallucinate non-existent propagation edges across all models tested. These results establish that outcome-based evaluation alone is insufficient for assessing LLM reasoning in complex system diagnostics. Process-level ground truth is a necessary foundation for developing agents whose explanations can be trusted to guide remediation in practice.

References

Deathstarbench. `https://github.com/delimitrou/DeathStarBench/tree/master`, 2024. Accessed: 2026-02-10.

Online boutique. `https://github.com/GoogleCloudPlatform/microservices-demo`, 2024. Accessed: 2026-02-10.

Gaia dataset. `https://github.com/CloudWise-OpenSource/GAIA-DataSet`, 2024. Accessed: 2026-02-10.

Sock shop. `https://github.com/microservices-demo/microservices-demo`, 2024. Accessed: 2024-08-21.

Y. Chen, J. Pan, J. Clark, Y. Su, N. Zheutlin, B. Bhavya, R. Arora, Y. Deng, S. Jha, and T. Xu. Stratus: A multi-agent system for autonomous reliability engineering of modern clouds. *arXiv preprint arXiv:2506.02009*, 2025a.

Y. Chen, M. Shetty, G. Somashekar, M. Ma, Y. Simmhan, J. Mace, C. Bansal, R. Wang, and S. Rajmohan. Aiopslab: A holistic framework to evaluate ai agents for enabling autonomous clouds. *arXiv preprint arXiv:2501.06706*, 2025b.

A. Fang, S. Zhang, Y. Yang, H. Wu, J. Xu, X. Wang, R. Wang, M. Wang, Q. Lu, and P. He. Rethinking the evaluation of microservice rca with a fault propagation-aware benchmark. *arXiv preprint arXiv:2510.04711*, 2025.

X. Han, S. Absar, L. Zhang, and S. Yuan. Root cause analysis of anomalies in multivariate time series through granger causal discovery. In *The Thirteenth International Conference on Learning Representations*, 2025.

A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu. Root cause analysis of failures in microservices through causal discovery. *Advances in Neural Information Processing Systems*, 35:31158–31170, 2022.

Z. Li, N. Zhao, S. Zhang, Y. Sun, P. Chen, X. Wen, M. Ma, and D. Pei. Constructing large-scale real-world benchmark datasets for aiops. *arXiv preprint arXiv:2208.03938*, 2022.

H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.

P. Lin. Self-balancing agentic AI: Test-time diffusion and context engineering re-imagined for deep research. `https://github.com/thinkdepthai/Deep_Research`, 2025.

L. Luo, Y. Liu, R. Liu, S. Phatale, M. Guo, H. Lara, Y. Li, L. Shu, Y. Zhu, L. Meng, et al. Improve mathematical reasoning in language models by automated process supervision. *arXiv preprint arXiv:2406.06592*, 2024.

J. Pearl. *Causality*. Cambridge university press, 2009.

L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang. Rcaeval: A benchmark for root cause analysis of microservice systems with telemetry data. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 777–780, 2025.

P. Spirtes, C. N. Glymour, and R. Scheines. *Causation, prediction, and search*. MIT press, 2000.

J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang. Openrca: Can large language models locate the root cause of software failures? In *The Thirteenth International Conference on Learning Representations*, 2025.

Y. Yan, J. Su, J. He, F. Fu, X. Zheng, Y. Lyu, K. Wang, S. Wang, Q. Wen, and X. Hu. A survey of mathematical reasoning in the era of multimodal large language model: Benchmark, method & challenges. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11798–11827, 2025.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

Z. Yin, Q. Sun, Z. Zeng, Q. Cheng, X. Qiu, and X.-J. Huang. Dynamic and generalizable process reward modeling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4203–4233, 2025.

G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, and X. Li. Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments. In *Proceedings of the Web Conference 2021*, pages 3087–3098, 2021.

G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng. Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 553–565, 2023.

T. Zeng, S. Zhang, S. Wu, C. Classen, D. Chae, E. Ewer, M. Lee, H. Kim, W. Kang, J. Kunde, et al. Versaprm: Multi-domain process reward model via synthetic reasoning data. In *Forty-second International Conference on Machine Learning*.

L. Zhang, Y. Zhai, T. Jia, C. Duan, S. Yu, J. Gao, B. Ding, Z. Wu, and Y. Li. Thinkfl: Self-refining failure localization for microservice systems via reinforcement fine-tuning. *arXiv preprint arXiv:2504.18776*, 2025a.

S. Zhang, A. Fang, Y. Yang, R. Cheng, X. Tang, and P. He. Dynacausal: Dynamic causality-aware root cause analysis for distributed microservices. *arXiv preprint arXiv:2510.22613*, 2025b.

X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering*, 47(2):243–260, 2018.

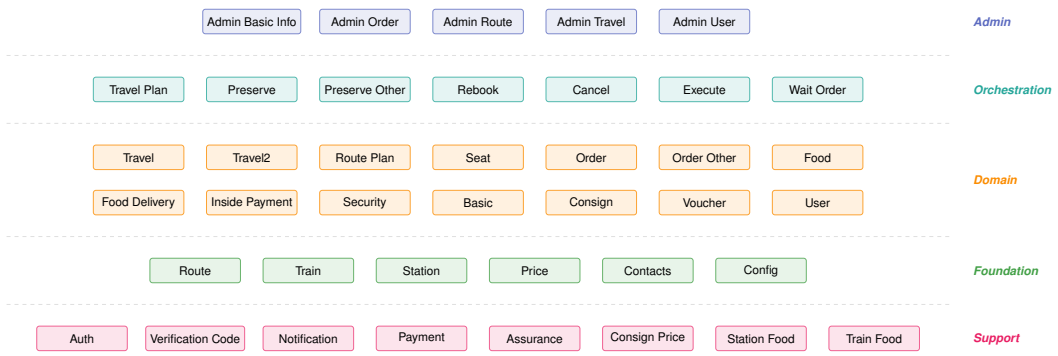


Figure 4: Service topology of the TrainTicket benchmark system, comprising 40 microservices organized in a layered architecture: **Admin** services manage system entities, **Orchestration** services implement business workflows, **Domain** services encapsulate core logic, **Foundation** services provide basic data lookups, and **Support** services handle cross-cutting concerns. Services communicate via synchronous HTTP/RPC calls; lower-layer services are shared across multiple upstream callers, creating complex fan-in dependency patterns that make root cause localization challenging.

A Benchmark Systems and Topology

This section details the microservice systems used in our benchmark, including their architecture, scale, and the service dependency topology that defines the structural prior $\mathcal{G}$ for causal path extraction.

A.1 System Overview

Our benchmark is built on TrainTicket Zhou et al. [2018], a large-scale microservice system for train ticket booking comprising 44 services, 93 dependency edges, and a maximum call-chain depth of 6, backed by MySQL. We chose TrainTicket over other widely used benchmarks such as DeathStarBench Dea [2024], OnlineBoutique Onl [2024], MicroSS gai [2024], and SockShop soc [2024] because it offers the largest service count and the deepest request chains among open-source microservice testbeds, providing richer fault propagation scenarios for evaluating causal reasoning.

A.2 TrainTicket Service Dependency Topology

As shown in Figure 4, TrainTicket's 44 microservices are organized in a layered architecture, with RabbitMQ for asynchronous messaging and a shared MySQL instance.

The system exhibits several properties that make RCA challenging:

- **Deep call chains**: Booking operations traverse up to 6 service layers (Entry $\rightarrow$ Gateway $\rightarrow$ Orchestration $\rightarrow$ Domain $\rightarrow$ Foundation $\rightarrow$ Infrastructure).
- **High fan-in**: Foundation services (`Station`, `Route`, `Train`) are shared by many upstream callers, creating complex dependency patterns where a single failure can manifest in multiple symptom locations.
- **Shared infrastructure**: 23 services connect to a shared MySQL 5.7 instance, introducing vertical propagation paths from database-level faults to application-level errors.
- **Circular dependencies**: Some services exhibit mutual dependencies (e.g., `Travel` $\leftrightarrow$ `Seat`) that require temporal unrolling for causal analysis.

B Root Cause Taxonomy

This section formalizes the ground truth label schema and provides the complete fault type catalog.

Table 4: Ground truth label schema. Each fault injection instance is annotated with a structured label across multiple granularity levels. The "Cardinality" column shows the number of possible values per field in the TrainTicket system.

FIELD	DESCRIPTION	CARDINALITY	EXAMPLE VALUE
`service`	TARGET MICROSERVICE	44	`ts-order-service`
`pod`	TARGET KUBERNETES POD	$\sim$80	`ts-order-service-7b8f4-xk9z2`
`container`	TARGET CONTAINER WITHIN POD	$\sim$80	`ts-order-service`
`function`	TARGET JAVA METHOD (JVM FAULTS)	$\sim$500	`com.order.service.impl.create`
`span`	TARGET TRACE SPAN (HTTP FAULTS)	$\sim$200	`POST /api/v1/orderservice/order`
`metric`	ANOMALOUS GOLDEN SIGNAL	$\sim$30	`http.server.request.duration`

B.1 Ground Truth Schema

Each fault injection instance is annotated with a structured ground truth label that specifies the root cause at multiple granularity levels. Table 4 shows the schema fields and their cardinality in the TrainTicket system.

The multi-level annotation enables evaluation at different abstraction levels. Service-level evaluation ($|\mathcal{V}| = 44$) is the most commonly reported granularity in prior work. Finer-grained levels (function, span) provide more precise localization but increase label space complexity substantially.

B.2 Complete Fault Type Catalog

Table 5 presents all 25 fault types implemented in our chaos engineering framework, organized by Chaos Mesh CRD category. Each fault type is classified by:

- **Target layer**: Whether the fault targets infrastructure resources (Pod, network, CPU/memory) or application logic (HTTP endpoints, JVM methods, database queries).
- **Applicable ground truth levels**: Which fields of the ground truth schema are relevant. Infrastructure faults are labeled at the `service`/`pod` level, while application faults include `function` or `span` annotations.
- **Propagation channel**: Whether the fault propagates horizontally (between logical entities via RPC) or vertically (from infrastructure to logical layers via resource contention), or both.

C LLM-Based RCA Agent Details

This section details the evaluation framework, agent architecture, and prompt design for the LLM-based RCA agents evaluated in our benchmark.

C.1 Agent Evaluation Framework

Each LLM agent receives a datapack containing multimodal telemetry (traces, metrics, logs in parquet format) and produces a structured CausalGraph: a directed graph with annotated root causes, affected nodes, and causal edges. Evaluation operates at two levels: outcome-level (root cause accuracy) and process-level (causal edge and node fidelity against ground truth). This dual evaluation design enables the stratified Edge F1 analysis presented in Section 3.2.

C.2 Agent Architecture

All agents follow a tool-augmented LLM architecture adapted from the Deep Research agent Lin [2025], operating in a ReAct-style Yao et al. [2022] investigation loop with three nodes:

1. `llm_call` (Decision Node): The LLM analyzes the current investigation state—consisting of the incident description, prior tool observations, and its own reflections—and either issues additional tool calls or terminates the loop.
2. `tool_node` (Execution Node): Dispatches all tool calls from the LLM response, executes them asynchronously, and returns observations as `ToolMessage` objects. Errors are caught and returned as observations (the loop does not crash on tool failures).

Table 5: Complete fault type taxonomy with 31 fault types across 7 Chaos Mesh CRD categories. Each category targets a different system abstraction layer. The "Applicable Levels" column indicates which ground truth fields are relevant for each fault type. Infrastructure-level faults (Pod/Stress/Network) affect resource metrics, while application-level faults (HTTP/JVM) target specific code paths and API endpoints.

Category	Fault Type	Parameters	Target Layer	Applicable Levels	Propagation Channel
PodChaos	PodKill	Kill signal (SIGKILL)	Infrastructure	service, pod	Vertical
	PodFailure	Unavailability duration	Infrastructure	service, pod	Vertical
	ContainerKill	Target container	Infrastructure	service, pod, container	Vertical
StressChaos	CPUStress	Core count, load %	Infrastructure	service, pod	Vertical
	MemoryStress	Memory size (MB)	Infrastructure	service, pod	Vertical
HTTPChaos	HTTPRequestAbort	Abort code, target path	Application	service, span	Horizontal
	HTTPResponseAbort	Abort code, target path	Application	service, span	Horizontal
	HTTPRequestDelay	Delay (ms), target path	Application	service, span	Horizontal
	HTTPResponseDelay	Delay (ms), target path	Application	service, span	Horizontal
	HTTPResponseReplaceBody	Body type {empty, random}	Application	service, span	Horizontal
	HTTPResponsePatchBody	Patch content	Application	service, span	Horizontal
	HTTPRequestReplacePath	New path	Application	service, span	Horizontal
	HTTPRequestReplaceMethod	Method {GET, POST, ...}	Application	service, span	Horizontal
	HTTPResponseReplaceCode	Status code	Application	service, span	Horizontal
DNSChaos	DNSError	Target hostname	Infrastructure	service, pod	Vertical
	DNSRandom	Target hostname	Infrastructure	service, pod	Vertical
NetworkChaos	NetworkDelay	Delay (ms), jitter	Infrastructure	service, pod	Vertical + Horizontal
	NetworkLoss	Loss rate (%)	Infrastructure	service, pod	Vertical + Horizontal
	NetworkDuplicate	Duplicate rate (%)	Infrastructure	service, pod	Vertical + Horizontal
	NetworkCorrupt	Corrupt rate (%)	Infrastructure	service, pod	Vertical + Horizontal
	NetworkBandwidth	Rate (bps)	Infrastructure	service, pod	Vertical + Horizontal
	NetworkPartition	Partition direction	Infrastructure	service, pod	Vertical + Horizontal
TimeChaos	TimeSkew	Offset (ms)	Infrastructure	service, pod	Vertical
JVMChaos	JVMLatency	Delay (ms), target class.method	Application	service, function	Horizontal
	JVMReturn	Return value, type {String, Int}	Application	service, function	Horizontal
	JVMException	Exception class, target method	Application	service, function	Horizontal
	JVMGarbageCollector	—	Application	service, pod	Vertical
	JVMCPUStress	CPU count	Application	service, pod	Vertical
	JVMMemoryStress	Type {heap, stack}, size	Application	service, pod	Vertical
	JVMMySQLLatency	Delay (ms), target method	Application	service, function	Horizontal
	JVMMySQLException	Exception, target method	Application	service, function	Horizontal

3. `compress_rca_findings` (Synthesis Node): Invoked when the LLM stops issuing tool calls. A separate LLM call synthesizes all accumulated evidence into a structured CausalGraph JSON.

The control flow follows:

$$\texttt{START} \rightarrow \texttt{llm_call} \xrightarrow{\text{tool_calls?}} \texttt{tool_node} \rightarrow \texttt{llm_call} \rightarrow \cdots \rightarrow \texttt{compress} \rightarrow \texttt{END}$$

Tool suite. The agent has access to four tools:

- `list_tables_in_directory`: Discovers available parquet files and their metadata (row/column counts).
- `get_schema`: Inspects the column names, types, and row count of a parquet file before querying.
- `query_parquet_files`: Executes SQL queries over parquet files (supports SELECT, WHERE, JOIN, aggregations).
- `think_tool`: A reflection tool that records the agent's internal reasoning. The prompt instructs the agent to call this tool after each query to summarize findings and plan the next investigation step.

The prompt recommends a tool call budget of 10–15 calls for typical incidents and suggests stopping after 20, but no hard ceiling is enforced—the LangGraph recursion limit is set to 3,000 (effectively unbounded), so models are free to conduct longer investigations when needed. All models use the same tool definitions and identical API retry logic (exponential backoff with max 5 retries).

C.3 CausalGraph Output Schema

Every agent must produce a single JSON object conforming to the `CausalGraph` schema below. This unified format enables fully automated evaluation at both the outcome and process levels.

SCHEMA CausalGraph JSON

```
{
  "nodes": [
    {"component": "ts-order-service",
     "state": ["HIGH_ERROR_RATE"],
```

```
      "timestamp": 1718234567000000000},
    {"component": "ts-travel-service",
     "state": ["TIMEOUT"],
     "timestamp": 1718234569000000000},
    {"component": "ts-station-service",
     "state": ["UNAVAILABLE"],
     "timestamp": 1718234565000000000}
  ],
  "edges": [
    {"source": "ts-station-service",
     "target": "ts-travel-service"},
    {"source": "ts-travel-service",
     "target": "ts-order-service"}
  ],
  "root_causes": [
    {"component": "ts-station-service",
     "state": ["UNAVAILABLE"],
     "timestamp": 1718234565000000000}
  ],
  "component_to_service": {}
}
```


The four top-level fields are:

- `nodes` — All services/components involved in the incident. Each node carries:
  - `component`: entity identifier (e.g., service name, pod name, span path).
  - `state`: list of abnormal states drawn from the standardized vocabulary (Table 6).
  - `timestamp` (optional): Unix timestamp in nanoseconds marking the onset of the anomaly.
- `edges` — Directed causal edges. An edge (`source`, `target`) means the failure of `source` *caused* the failure of `target`.
- `root_causes` — A subset of `nodes` identifying the origin(s) of the incident. This is the primary field for outcome-level evaluation (Pass@1).
- `component_to_service` — Optional mapping from fine-grained component names (spans, pods) to their parent service name, enabling evaluation at multiple granularity levels.

In the example above, `ts-station-service` became `UNAVAILABLE` at t_0, causing `ts-travel-service` to `TIMEOUT` at t_1, which in turn triggered `HIGH_ERROR_RATE` in `ts-order-service` at t_2—a three-hop propagation path with a single root cause. Both the ground-truth labels and agent predictions use this identical schema, so evaluation reduces to set comparisons over nodes, edges, and root causes (Section D).

C.4 Prompt Design

Each agent receives a two-phase prompt. In the *investigation phase*, a system prompt (`RCA_ANALYSIS_SP`) and a user prompt (`RCA_ANALYSIS_UP`) define the task, available tools, analysis strategy, and output schema. In the *synthesis phase*, a separate prompt pair (`COMPRESS_FINDINGS_SP/UP`) instructs the model to distill all tool observations into a structured CausalGraph. The complete prompts are reproduced below.

C.4.1 Investigation Phase

SYSTEM: RCA_ANALYSIS_SP

```
You are a Root Cause Analysis (RCA) expert conducting systematic investigation of
system incidents.

For context, today's date is {date}.

<Task>

Your goal is to identify:

1.  Root Cause Service:  Which service is the origin of the failure
```

2. Fault Propagation Path: How the error propagated through the system as a causal graph

You will analyze telemetry data (logs, traces, metrics) to construct a complete picture of the incident.

<Available Data Types>

The input data consists of parquet files with three main types:

1. Logs: normal_logs.parquet, abnormal_logs.parquet
2. Traces: normal_traces.parquet, abnormal_traces.parquet
3. Metrics: normal_metrics.parquet, abnormal_metrics.parquet

<Available Tools>

1. query_parquet_files: Query parquet files using SQL syntax for data analysis.
2. list_tables_in_directory: List all parquet files in a directory with metadata.
3. get_schema: Get schema information of a parquet file.

CRITICAL: Use think_tool after each search to reflect on results and plan next steps.

<Analysis Instructions>

Think like an experienced SRE investigating a production incident:

1. Understand the Incident - Read the incident description carefully
2. Discover Data Sources - Use list_tables_in_directory
3. Explore Data Schema - Use get_schema to understand data structure
4. Query for Evidence - Use query_parquet_files to extract relevant information
5. Identify Root Cause - Determine which service initiated the failure
6. Map Propagation Path - Build the causal graph showing how the error spread

<Hard Limits>

Tool Call Budget:

- Use 10-15 tool calls for typical incidents
- Stop after 20 tool calls if you cannot find conclusive evidence

Stop Immediately When:

- You can identify the root cause service with confidence
- You have mapped the fault propagation path
- You have sufficient evidence from logs/traces/metrics

<Output Requirements>

Final output MUST be a structured JSON with CausalGraph format containing:

1. nodes: List of CausalNode objects (component, state, timestamp)
2. edges: List of CausalEdge objects (source, target)
3. root_causes: List of CausalNode objects identifying root cause(s)
4. component_to_service: Mapping from component names to service names

Available States: service (HEALTHY, HIGH_ERROR_RATE, HIGH_LATENCY, UNAVAILABLE), span (HIGH_P99_LATENCY, TIMEOUT, HIGH_LOG_ERROR, ...), pod (KILLED, PROCESS_PAUSED, HIGH_CPU, NETWORK_DELAY, ...), deployment (AVAILABLE, DEGRADED, FAILED, UNKNOWN)

USER: RCA_ANALYSIS_UP

Please conduct a Root Cause Analysis for the following incident:

Incident Description

{incident_description}

Your Mission

Identify:

1. Root Cause Service: The service where the failure originated

2. Fault Propagation Graph: The complete causal chain from root cause to all affected services

Investigation Strategy

1. Discover Available Data - Use list_tables_in_directory

2. Understand Data Structure - Use get_schema on key files

3. Identify Anomalies - Query abnormal data vs normal data; find: WHEN and WHICH services

4. Trace Service Dependencies - Use trace data for call chains

5. Determine Root Cause - Find the earliest abnormal service; verify it is the origin

6. Map Propagation Path - Build directed edges following the causal chain

Output Format

CausalGraph JSON with nodes, edges, root_causes, component_to_service.

Use standardized state values: HIGH_ERROR_RATE, HIGH_LATENCY, TIMEOUT, HIGH_CPU, NETWORK_DELAY, KILLED, etc.

Remember: Use think_tool after each query. Stop when you have enough evidence. Base all conclusions on actual data, not assumptions.

C.4.2 Synthesis Phase

SYSTEM: COMPRESS_FINDINGS_SP

You are an expert Root Cause Analysis synthesizer.

Your task is to convert investigation findings into structured CausalGraph JSON format.

For context, today's date is {date}.

USER: COMPRESS_FINDINGS_UP

You are an RCA expert who has conducted a thorough investigation of a system incident.

Synthesize all findings into a structured CausalGraph JSON format.

<Task>

Transform all investigation findings from tool calls into a structured CausalGraph:

1. Root Cause: Which service(s) initiated the failure

2. Propagation Path: How the fault spread (as a directed graph)

3. All Affected Services: Complete list of impacted services

<Tool Call Filtering>

- Include: All query results showing anomalies, errors, failures

- Exclude: think_tool calls (internal reasoning)

- Focus on: Concrete evidence of what went wrong and how it propagated

<Output Requirements>

Output ONLY a valid JSON object in CausalGraph format:

{"nodes": [...], "edges": [...], "root_causes": [...], "component_to_service": {...}}

Available States by Component Type:

- pod: KILLED, PROCESS_PAUSED, HIGH_CPU, HIGH_MEMORY, NETWORK_DELAY, NETWORK_LOSS, ...

- container: KILLED, PROCESS_PAUSED, HIGH_CPU, HIGH_MEMORY, ...

- service: HEALTHY, HIGH_ERROR_RATE, HIGH_LATENCY, UNAVAILABLE

```
- span:  HIGH_P99_LATENCY, HIGH_AVG_LATENCY, TIMEOUT, HIGH_LOG_ERROR,
CONNECTION_RESET, ...
- deployment/replica_set:  AVAILABLE, DEGRADED, FAILED, UNKNOWN
<Critical Rules>
- Output ONLY the JSON object, no markdown, no explanations
- The root_causes field is MANDATORY and must identify the actual root cause
service
- Service names must match those found in the investigation data
INVESTIGATION TOPIC: {incident_description}
Based on ALL the investigation messages above, synthesize your findings into the
CausalGraph JSON format.  Output the JSON object NOW:
```

C.4.3 Key Design Choices

- **Standardized state vocabulary**: To enable automated evaluation, the prompt defines an enumerated set of valid states for each entity type (Table 6), with a mapping guide (e.g., "error rate spike → `HIGH_ERROR_RATE`").
- **No topology leakage**: The agent is *not* provided with the service dependency graph. It must discover service dependencies from trace data, testing genuine investigative ability rather than pattern matching over a known graph.
- **Paired normal/abnormal data**: Both normal-period and abnormal-period telemetry are available, enabling the agent to perform differential analysis (e.g., comparing normal vs. abnormal error rates).
- **Structured output schema**: The CausalGraph format (`nodes`, `edges`, `root_causes`, `component_to_service`) is defined in both the investigation and synthesis prompts to ensure consistent output across models.

Table 6: Standardized state vocabulary for CausalGraph node annotations. Agents must use these exact values to enable automated evaluation via string and semantic matching.

ENTITY TYPE	AVAILABLE STATES
SERVICE	HEALTHY, HIGH_ERROR_RATE, HIGH_LATENCY, UNAVAILABLE
SPAN	HIGH_P99_LATENCY, HIGH_AVG_LATENCY, HIGH_ERROR_RATE, TIMEOUT, HIGH_LOG_ERROR, CONNECTION_RESET, MALFORMED_RESPONSE
POD	KILLED, PROCESS_PAUSED, HIGH_CPU, HIGH_MEMORY, NETWORK_DELAY, NETWORK_LOSS, NETWORK_PARTITION, DNS_ERROR, ...
CONTAINER	KILLED, PROCESS_PAUSED, HIGH_CPU, HIGH_MEMORY, ...
DEPLOYMENT	AVAILABLE, DEGRADED, FAILED, UNKNOWN

C.5 Model Configurations

- Claude Sonnet 4.5: `claude-sonnet-4-5-20250929`, max tokens 128k
- GPT-5.1: `gpt-5.1`, max tokens 128k
- K2: `ep-20250919144417-j2j4b`, max tokens 128k
- GLM-4.7-Flash: `zai-org/GLM-4.7-Flash`, max tokens 128k
- Seed 1.6: `ep-20251110181330-f8sjl`, max tokens 128k
- Qwen3-Next-80B: `qwen3-next-80b-a3b-instruct`, max tokens 128k
- Qwen3-32B: `qwen3-32b`, max tokens 128k

All models use identical system prompts, tool definitions, and temperature (0.7) for reproducibility. Each model is evaluated on the same prompt template and tool suite; minor differences in instance counts across models arise from API failures and timeout-induced retries during evaluation runs.

D Evaluation Protocol and Metric Definitions

We organize our evaluation metrics into three families: *outcome-level* (Pass@1 Accuracy, assessing root cause identification), *process-level* (evaluating causal reasoning quality), and *dataset difficulty* (characterizing the inherent diagnostic difficulty of each instance).

D.1 Outcome-Level Metric

We use a single outcome-level metric that directly measures whether the agent correctly identifies the root cause on its first prediction. Let $\hat{\mathcal{R}} = \{(\ell_i, n_i, r_i)\}_{i=1}^{N}$ be the ranked list of predicted root causes, where ℓ_i is the evaluation level, n_i is the entity name, and r_i is the rank. Let $\mathcal{R}^*$ be the ground truth root cause set, and $\mathcal{Q}$ the set of all evaluation queries (datapacks).

Pass@1 Accuracy.

$$\text{Pass@1} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{1}\left[(\ell_1, n_1) \in \mathcal{R}_q^*\right] \tag{3}$$

The fraction of cases where the agent's prediction is a correct root cause. This is a strict metric: the agent must identify the right answer on its first attempt.

D.2 Process-Level Metrics

These metrics evaluate the quality of the agent's causal reasoning by comparing the predicted CausalGraph against the ground-truth propagation graph $\mathcal{G}^*$. We decompose the evaluation into *primary* metrics (service-level, deterministic matching) and *secondary* metrics (component-level, with LLM-assisted semantic matching for name normalization).

D.2.1 Primary Metrics (Service-Level)

All primary metrics operate on service-level representations. Service names are normalized (lower-cased, `ts-` prefix stripped, hyphens removed) before comparison.

Edge F1. Let $\hat{\mathcal{E}}$ and $\mathcal{E}^*$ be the service-level edge sets of the predicted and ground-truth causal graphs, respectively:

$$\text{Edge Precision} = \frac{|\hat{\mathcal{E}} \cap \mathcal{E}^*|}{|\hat{\mathcal{E}}|}, \quad \text{Edge Recall} = \frac{|\hat{\mathcal{E}} \cap \mathcal{E}^*|}{|\mathcal{E}^*|} \tag{4}$$

Edge F1 is the harmonic mean. This is the most important process-level metric, as it directly measures whether the agent correctly identified causal relationships between services.

Node F1. Let $\hat{\mathcal{V}}$ and $\mathcal{V}^*$ be the service-level node sets:

$$\text{Node Precision} = \frac{|\hat{\mathcal{V}} \cap \mathcal{V}^*|}{|\hat{\mathcal{V}}|}, \quad \text{Node Recall} = \frac{|\hat{\mathcal{V}} \cap \mathcal{V}^*|}{|\mathcal{V}^*|} \tag{5}$$

Node F1 measures whether the agent correctly identified the set of affected services.

Root Cause F1. Let $\hat{R}$ and R^* be the predicted and ground-truth root cause service sets:

$$\text{RC Precision} = \frac{|\hat{R} \cap R^*|}{|\hat{R}|}, \quad \text{RC Recall} = \frac{|\hat{R} \cap R^*|}{|R^*|} \tag{6}$$

Root Cause F1 supports multiple root causes per incident. This metric is complementary to the outcome-level Pass@1: while Pass@1 measures "did you find *any* correct root cause?", RC F1 penalizes both missed and hallucinated root causes.

Boundary case. When both the predicted and ground-truth sets are empty for a metric (e.g., both graphs have zero edges), we assign a perfect score of 1.0, since the agent correctly predicted the absence of that element.

D.2.2 Secondary Metrics (Component-Level)

Secondary metrics operate at finer granularity (spans, pods, containers) using the raw component identifiers from the CausalGraph.

Component F1. Matches predicted components against ground-truth components. Because naming conventions vary (e.g., `ts-order-service` vs. `order-service` vs. `OrderService`), matching uses two stages: (1) exact match after normalization, then (2) LLM-based semantic matching for remaining unmatched pairs.

State Coverage. For components appearing in both graphs, measures the fraction where the predicted state set semantically matches the ground-truth state set (e.g., `slow` $\approx$ `HIGH_LATENCY`). LLM-based matching (GPT-4o-mini) is used for semantic equivalence.

Hallucination Rate.

$$\text{HR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} |\hat{\mathcal{E}}_q \setminus \mathcal{E}_q^*| \tag{7}$$

The average number of fabricated causal edges per diagnosis. Edges in the agent's graph that have no counterpart in the ground truth are counted as hallucinations.

Stratified Edge F1. To measure whether correct diagnoses are supported by faithful causal reasoning, we report Edge F1 separately for correctly diagnosed instances ($\mathcal{Q}^+$) versus incorrectly diagnosed instances ($\mathcal{Q}^-$):

$$\text{Edge F1 } (+) = \frac{1}{|\mathcal{Q}^+|} \sum_{q \in \mathcal{Q}^+} \text{Edge F1}_q \tag{8}$$

$$\text{Edge F1 } (-) = \frac{1}{|\mathcal{Q}^-|} \sum_{q \in \mathcal{Q}^-} \text{Edge F1}_q \tag{9}$$

where $\mathcal{Q}^+ = \{q \in \mathcal{Q} : \text{Pass@1}_q = 1\}$ and $\mathcal{Q}^- = \mathcal{Q} \setminus \mathcal{Q}^+$. A low Edge F1 (+) implies that even when the root cause is correctly identified, the agent's underlying causal reasoning is flawed (a "lucky guess").

Path Reachability (PR). Path Reachability measures the fraction of all instances where the agent both correctly identifies the root cause *and* constructs at least one valid directed path from that root cause to a ground-truth symptom node in its predicted graph:

$$\text{PR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{1}\left[q \in \mathcal{Q}^+ \ \wedge \ \exists \text{ path } \hat{r}_q \rightsquigarrow \hat{s}_q \text{ in } \hat{\mathcal{G}}_q \mid \hat{r}_q \in R_q^*,\ \hat{s}_q \in A_q^*\right] \tag{10}$$

where R_q^* and A_q^* are the ground-truth root cause and symptom service sets for instance q. Incorrect diagnoses receive PR $= 0$ by definition, since the wrong root cause cannot anchor a valid propagation path. PR is therefore strictly upper-bounded by Pass@1, and the gap between the two quantifies how often a correct diagnosis lacks even a single actionable propagation path.

D.3 Dataset Difficulty Metrics

To enable stratified analysis (e.g., "how does agent performance degrade as diagnostic difficulty increases?"), we characterize each fault injection instance using the metrics in Table 7. All metrics are derived from the ground-truth causal graph $\mathcal{G}^*$ and require no LLM involvement, ensuring they are independent of the agent under evaluation.

We exclude 5 instances where the fault is injected directly at the entry-point service (`ts-ui-dashboard`, SPL $= 0$), as these cases involve no inter-service propagation and do not test multi-hop causal reasoning. After filtering, 1232 diagnostic instances remain.

Empirical validation. We verify that these metrics genuinely predict diagnostic difficulty by examining the monotonic relationship between each metric and agent performance across all 7 models. Service Path Length (SPL) is the single strongest predictor: average Pass@1 drops from 66.7% at SPL $= 1$ to 24.6% at SPL $= 5$. Fault category also matters: agents achieve 61.2% Pass@1 on network faults but only 37.9% on pod-level faults (container/pod kill and failure), reflecting the difference between localized, deterministic faults and diffuse, resource-contention faults.

Table 7: Dataset difficulty characterization metrics derived from the ground-truth causal graph $\mathcal{G}^*$. All metrics are computed prior to agent evaluation and require no LLM involvement. We exclude trivial cases where the fault is injected directly at the entry service (SPL $= 0$), retaining 1232 diagnostic instances.

METRIC	FORMULA	RANGE	INTERPRETATION
SPL	$\max_{r \in R} d_{\text{svc}}(r, \text{SYMPTOM})$	1–5	SERVICE-LEVEL PATH LENGTH FROM ROOT CAUSE TO THE FARTHEST AFFECTED ENTRY-POINT SERVICE. MEASURES THE DEPTH OF MULTI-HOP REASONING REQUIRED.
N_{SVC}	$\lvert\{v \in \mathcal{V}^*_{\text{svc}}\}\rvert$	2–13	NUMBER OF DISTINCT SERVICES IN $\mathcal{G}^*$. MORE SERVICES $\Rightarrow$ LARGER SEARCH SPACE FOR THE AGENT.
N_{EDGE}	$\lvert\mathcal{E}^*_{\text{svc}}\rvert$	1–22	NUMBER OF CROSS-SERVICE CAUSAL EDGES IN $\mathcal{G}^*$. MORE EDGES $\Rightarrow$ MORE COMPLEX PROPAGATION TOPOLOGY.
FAULT TYPE	CATEGORICAL	7 CLASSES	CHAOS MESH CRD CATEGORY: PODCHAOS, STRESSCHAOS, HTTPCHAOS, DNSCHAOS, NETWORKCHAOS, TIMECHAOS, JVMCHAOS. DIFFERENT TYPES PRODUCE QUALITATIVELY DIFFERENT SYMPTOMS.

These metrics enable controlled experiments such as: plotting Pass@1 vs. SPL to reveal whether agents struggle with multi-hop reasoning (Section 3.2), or stratifying by fault type to identify which fault categories expose fundamental reasoning gaps.

E Experimental Details

E.1 Cluster Configuration

Our testbed runs on Kubernetes v1.29 with the following specifications:

- **Cluster**: 1 control-plane node (48 vCPU, 64GB RAM) and 6 worker nodes (32 vCPU, 64GB RAM each), running Debian 12 with containerd 1.7.5
- **Microservices**: 44 services deployed via Helm, predominantly Java 17 / Spring Boot 3.2.0 with additional services in Python, Go, and JavaScript
- **Resource allocation**: Each service requests 1 CPU / 1GB RAM with limits of 5 CPU / 3GB RAM; HPA enabled (1–3 replicas, 200% CPU target); JVM heap set to 2GB with G1GC
- **Traffic Load**: Sustained load via 2-thread load generator (100ms sleep interval) with Poisson-distributed user sessions
- **Monitoring Stack**: OpenTelemetry Kube Stack with per-node daemon collectors and a central deployment collector; Prometheus with AlertManager (15s scrape interval); 100% trace sampling via OpenTelemetry auto-instrumentation (Elastic Java agent)
- **Fault Injection**: Chaos Mesh (custom fork with runtime mutator) with programmatic CRD generation
- **Database**: Shared MySQL 5.7 instance (max 1,024 connections); RabbitMQ 4.0.7 for asynchronous messaging

E.2 FORGE Hyperparameters

Key parameters for the FORGE annotation pipeline:

- **State Detection Window** Δ: 3 s for span/service-level metrics; 5 s for pod, container, and machine-level metrics
- **State Activation**: Dual detection—fixed utilization thresholds (>80% CPU/memory) combined with Z-score baseline comparison ($Z > 3\sigma$ for critical anomalies); adaptive multiplier thresholds for latency metrics based on coefficient of variation (CV)
- **Maximum Path Length**: 5 hops (default), with diamond-shaped revisits allowed (max 2 visits per node)
- **Rule Confidence**: Default 0.8 per propagation rule, with per-rule overrides

E.3 Propagation Rule Set

Table 8 lists the complete rule set $\mathcal{R}$ used in Phase 1 (Section 2.3). Rules are constructed by enumerating all (source entity type $\times$ source state $\times$ edge type $\times$ destination entity type) combinations in the system dependency graph, then retaining only those for which a known physical or logical mechanism exists. We group rules into four categories by injection granularity.

Vertical rules (container→span, pod→span) encode resource contention: a throttled container or pod degrades all spans of its co-located service through the Kubernetes scheduling hierarchy (Container $\subset$ Pod $\subset$ Node). **First-hop rules** (service→span) bridge the aggregation layer, since services are logical groupings without directly observable states. **Horizontal rules** (span→span) capture RPC call-chain cascades, the dominant propagation channel, accounting for over 80% of observed edges. **Cross-channel rules** (span→pod→span) model JVM stress scenarios where resource contention on a shared pod affects sibling services.

The rule set is intentionally over-inclusive: rules with zero empirical usage (e.g., service error→span) are retained for completeness, and Phase 2's statistical verification (Section 2.4) filters false positives. Rules lacking any empirical support after 500+ injection experiments were removed in version 2.0 (e.g., circuit-breaker bypass, pod-level disk/network faults), as noted in the version history.

Table 8: Complete propagation rule set $\mathcal{R}$. *Src*/*Dst* denote source and destination entity types; *Path* shows intermediate hops for multi-hop rules.

ID	Category	Src Kind : States	Path	Dst Kind : States	Mechanism
Vertical: Container → Span					
C-01	vertical	container : {cpu, mem}	pod → svc →	span : {lat, p99, timeout, missing, err}	Resource contention
C-02	vertical	container : {restart}	pod → svc →	span : {missing, err, timeout}	Process termination
Vertical: Pod → Span					
P-01	vertical	pod : {cpu, mem}	svc →	span : {lat, p99, timeout}	Resource contention
First-hop: Service → Span					
S-01	first-hop	service : {any}	—	span : {lat, p99, timeout}	Latency inheritance
S-02	first-hop	service : {err, unavail}	—	span : {err, timeout, reset, missing, log_err}	Error inheritance
Horizontal: Span → Span (RPC cascade)					
H-01	horizontal	span : {lat, p99}	—	span : {lat, p99, timeout}	Callee slowdown
H-02	horizontal	span : {err, timeout}	—	span : {err, timeout}	Error propagation
H-03	horizontal	span : {missing}	—	span : {err, timeout, missing}	Missing callee
H-04	horizontal	span : {err, missing}	healthy span →	span : {err, missing}	Controller bypass
H-05	horizontal	span : {err, timeout}	—	span : {timeout}	Retry-induced wait
H-06	horizontal	span : {inj_affected}	—	span : {err, lat, p99, timeout, missing}	Infra fault relay
H-07	horizontal	span : {healthy}	healthy span →	span : {err, lat, p99, missing}	JVM injection bypass
Cross-channel: Span → Pod → Span					
X-01	cross	span : {lat, p99}	svc → pod → svc →	span : {lat, p99, timeout}	JVM stress via shared pod

E.4 Ground Truth Annotation Guidelines

Verification Process. To validate the correctness of the automatically extracted causal paths (Section 2.4), we deploy a two-stage LLM-based verification agent that follows a structured Standard Operating Procedure (SOP) encoding expert domain knowledge. The agent receives each candidate propagation path along with the full telemetry context and applies a systematic 2-step protocol to every edge:

1. **Step 1: Context gathering.** The agent uses code-analysis tools (`verify_span_call_chain`, `search_code_by_span`) to determine the call mechanism between the source and target nodes (HTTP call, shared resource, same entity) and to identify error-handling patterns (try-catch, circuit breakers, retries) that may block propagation.
2. **Step 2: Anomaly verification.** The agent queries time-series metrics (`query_metric`), compares trace patterns (`compare_trace_patterns`), and searches logs (`search_logs_by_keywords`) for both source and target nodes. It compares the fault period against the baseline period and checks three conditions: (a) both nodes show metric degradation or fault-triggered activation, (b) the source anomaly onset precedes the target, and (c) the observed pattern is consistent with the propagation mechanism identified in Step 1.

Each edge receives a verdict of PASS (valid propagation), FAIL (false positive), or UNCERTAIN (insufficient evidence), with a structured note recording the evidence and reasoning. The verification proceeds in two stages:

- **False Positive detection**: Verifies all edges in the predicted paths in BFS order from the injection point, with level-wise parallelism (edges at the same BFS depth are verified concurrently). Each edge is allocated a budget of 10 tool calls.
- **False Negative detection**: Starting from the injection node, performs BFS exploration over the system topology to discover propagation paths that the extraction algorithm missed.

The verification agent uses Kimi-K2.5 as the backbone LLM (temperature 0.3, timeout 100s).

E.5 Scalability and Computational Cost

All experiments were conducted on a virtual machine with 32 CPU cores and 64GB RAM. The path extraction process for a single fault injection scenario, covering a 15-minute telemetry window ($\sim$1GB of metrics and traces), completes in an average of 3.5 minutes. Computational complexity scales with the number of active anomalous entities rather than total system components, demonstrating practical scalability.

E.6 Data Collection Protocol

For each fault injection experiment, we collect multimodal telemetry data organized into standardized parquet files:

- **Traces**: Distributed traces (normal + abnormal periods) with span-level attributes including service name, operation, duration, HTTP status codes, and parent-child relationships
- **Metrics**: Time-series metrics covering the four golden signals (latency, traffic, errors, saturation) from both Prometheus and OpenTelemetry exporters
- **Logs**: Structured log entries with severity levels and timestamps
- **Environment metadata**: Kubernetes pod/node mappings, resource limits, and injection configuration

Each datapack includes matched normal-period and abnormal-period data to enable the counterfactual analysis described in Section 2.4.

E.7 Anomaly Detection Thresholds

The per-node anomaly screen in Phase 2 (Section 2.4) uses three detector families, each matched to the distributional characteristics of the underlying metric.

Z-score detector (for approximately Gaussian metrics such as CPU and memory utilization). A metric value x is flagged if its Z-score $z = (x - \mu_{base})/\max(\sigma_{base}, \epsilon)$ exceeds a threshold, where μ_{base} and σ_{base} are the baseline mean and standard deviation and $\epsilon = 10^{-6}$ prevents division by zero. We use three severity levels: *critical* ($z > 3.0$), *warning* ($z > 2.0$), and *moderate* ($z > 1.5$).

Percentile detector (for heavy-tailed metrics such as error counts). Instead of parametric assumptions, anomalies are flagged relative to empirical baseline percentiles: *critical* if the value exceeds the baseline P99.9 or $2\times$P99; *warning* if it exceeds P99 or $1.5\times$P99; *moderate* if it exceeds P90 or $1.2\times$P99.

Adaptive latency detector (for request latency, which spans orders of magnitude across services). The threshold multiplier is $\tau = (\tau_{cv\text{-}base} + \tau_{cv\text{-}range} \cdot (1 - e^{-cv/\alpha})) \cdot (l_{ref}/l_{base})^{\beta}$, where cv is the baseline coefficient of variation, l_{base} the baseline mean latency, and l_{ref} a reference latency. This formulation encodes the principle that low-baseline services (e.g., 1 ms mean) tolerate large relative spikes with minimal user impact, while high-baseline services (e.g., 5 s mean) require tighter thresholds.

Design rationale. All three detectors are intentionally permissive at the per-node level: they aim to avoid false negatives (missing a genuinely affected node) at the cost of admitting some false positives (flagging coincidental deviations). False positives are subsequently eliminated by the conjunctive verification criterion: a candidate path is accepted only when every node passes the anomaly screen, every edge conforms to the propagation rule set $\mathcal{R}$, and downstream anomalies follow upstream causes in time. This layered design decouples the sensitivity of individual detectors from the overall precision of the ground truth.

Why conjunction filtering is effective. A candidate path π of length k is verified only if three independent conditions hold simultaneously at every hop: (i) statistical anomaly, (ii) structural rule conformance, and (iii) temporal ordering. Let p_s, p_r, and p_t denote the per-hop false-positive rates of each condition in isolation. Under an independence assumption, the probability that a spurious (non-causal) path of length k accidentally satisfies all three conditions at every hop is bounded by:

$$P(\text{false path accepted}) \;\leq\; (p_s \cdot p_r \cdot p_t)^k \tag{11}$$

Even with individually permissive thresholds (e.g., p_s=0.2, p_r=0.3, p_t=0.5), the per-hop joint false-positive rate is $0.2 \times 0.3 \times 0.5 = 0.03$, and for a typical path of length k=3 this yields $0.03^3 \approx 2.7 \times 10^{-5}$. This exponential decay in k means that longer propagation paths, which are harder to verify, are also the most resistant to spurious acceptance. The independence assumption is approximate: in practice the three conditions share some information (e.g., structural constraints partially determine temporal ordering). Nevertheless, the exponential scaling ensures robust filtering even when the conditions are moderately correlated.